

ASSIGNMENT 1

Name: Roshni Ghosh Tagore

Enrollment No: 12023002028175

Course: Python for Automation

Academic Year: 2023-2027

Department: Computer Science and Engineering (Artificial Intelligence & Machine Learning)

Campus: IEM Newtown

Problem Statement

Automate user interactions on web applications featuring isolated document frames (IFrames). The objective is to navigate to a target dynamic URL, locate an embedded iframe container, switch the execution focus into the iframe to edit content, and smoothly transfer the driver context back to the parent document root.

Objective

- To interact with nested HTML IFrames using Selenium WebDriver.
- To perform driver context switches using `driver.switch_to.frame()`.
- To manipulate input fields residing inside isolated iframe elements.
- To restore focus back to the top-level main window using `driver.switch_to.default_content()`.

Tools, Software, and Concepts Used

- **Language:** Python 3.x
- **Automation Suite:** Selenium WebDriver
- **IDE:** Visual Studio Code
- **Target Webpage:** <https://the-internet.herokuapp.com/iframe>
- **Key Concepts:** DOM Frame Context Switching, Element Locators (By.ID), Frame Navigation.

Source Code or Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://the-internet.herokuapp.com/iframe")
driver.maximize_window()

# Find the iframe
iframe = driver.find_element(By.ID, "mce_0_ifr")

# Switch to iframe
driver.switch_to.frame(iframe)

# Find the text area
text_box = driver.find_element(By.ID, "tinymce")

# Type directly
text_box.send_keys("Hello from Selenium")

time.sleep(3)

# Switch back to main page
driver.switch_to.default_content()

print("Switched back to main page")

driver.quit()
```

Output with Labelled Screenshots

```

assignment6.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  import time
4
5  driver = webdriver.Chrome()
6
7  driver.get("https://the-internet.herokuapp.com/iframe")
8  driver.maximize_window()
9
10 # Find the iframe
11 iframe = driver.find_element(By.ID, "mce_0_ifr")
12
13 # Switch to iframe
14 driver.switch_to.frame(iframe)
15
16 # Find the text area
17 text_box = driver.find_element(By.ID, "tinymce")
18
19 # Type directly
20 text_box.send_keys("Hello from Selenium")
21
22 time.sleep(3)
23
24 # Switch back to main page
25 driver.switch_to.default_content()
26
27 print("Switched back to main page")
28
29 driver.quit()

```

Figure 6.1: VS Code implementation of IFrame switching script (assignment6.py)

```

(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots\assignment6.py"
Switched back to main page
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots>

```

Figure 6.2: Terminal output confirming execution and return to main page context

Brief Result, Observation, and Conclusion

Result: The script successfully launched Chrome, switched execution context into the TinyMCE editor iframe, passed input text directly to the text editor, and seamlessly switched back to the main page context.

Observation: Dynamic web elements located inside an <iframe> element cannot be accessed directly from the main document scope. Attempting to locate them directly raises a `NoSuchElementException` unless explicit switching is performed via `driver.switch_to.frame()`.

Conclusion: Efficient context switching between main frames and sub-frames is essential for robust web scraping and test automation on complex modern web architectures.